

Recommender System using Amazon Reviews

Group 6: Hong Chang, Hema Kadali, Yerong Liu, Erin Murphy, Evan Wong

ABSTRACT: In this report, we demonstrate our solutions to purchase, category and rating prediction for a Kaggle competition. Some main concepts used in our models are popularity and Jaccard Similarity index for purchase prediction; collaborative filtering with latent factor modeling for rating prediction; NLP methods, text vectorization and classification with logistic regression, CNN for category prediction. This paper further discusses hyper-parameter tuning conducted to improve accuracy as well as our best approaches. At the end, our paper explores advanced methodologies for future research to embark on.

1. INTRODUCTION

We are building three recommender system use cases that can make predictions given reviews of clothing, shoes and jewelry on Amazon. We are provided with 200,000 Amazon reviews, including ratings, category labels and purchases. Specifically the three use cases aim to complete the following three tasks:

- Predict whether a user purchased a specific item
- Predict ratings for the (user,item) pairs
- Predict the category of the item from the review

We have conducted data analysis to understand the data patterns and used different methods as well as models to solve the three tasks.

2. DATA ANALYSIS

Below is an overview of the train and test datasets (Table 2.1). We will use the following key properties for prediction tasks: reviewerID, itemID, rating, reviewTime, categoryID, categories, and reviewText.

Train dataset	Purchase dataset
Total size: 200,000 Unique user: 39,239 Unique item: 19,914 Unique category: 0-4 Rating: 1-5 Sparsity: 99.97441%	Total size: 28,000 Unique user: 19,871 Unique item: 14,447 New user: 101 New item: 3
Rating dataset	Category dataset
Total size: 14,000 Unique user: 11,637 Unique item: 8,691 New user: 55 New item: 1	Total size: 14,000 Unique user: 11,698

Table 2.1 Property overview of train and test datasets

We also examined the data distribution along key properties that are critical to the models.

Popularity distribution of the products: We counted per product purchase history and plotted its histogram (Fig 2.1). We found most of the products were purchased less than ~20 times. A relatively small set

of products were purchased more than this threshold and could be considered as extremely popular items.

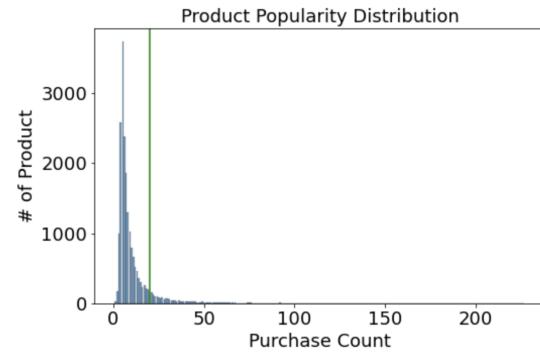


Figure 2.1 Product purchase count & distribution

Product average rating distribution: We found most of the products have an average rating in the range of 4 to 5 (Fig 2.2).

Product category distribution: In our analysis of the category distribution from products, most products were either in category 0 (women) or 1 (men) (Fig 2.3). This distribution was extremely skewed.

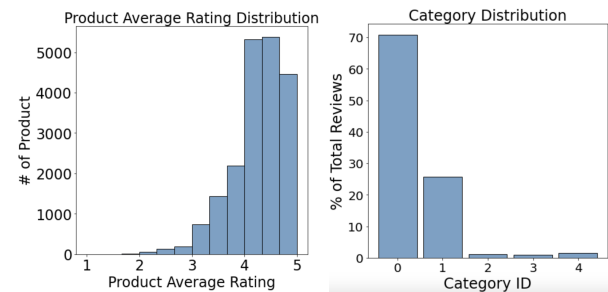


Figure 2.2, Figure 2.3 Avg. rating, Category distribution

3. METHODS & RESULTS

3.1 Purchase Prediction

Approach: Heuristic inferences that use popularity or similarity based methods are common for this type of prediction. The Popularity method assumes a user will always buy a “popular” item. Similarity methods

(Jaccard, cosine, Pearson) finds either user or item similarity and predicts whether a user will buy an item if the user bought a similar item before or if a similar user bought it before. Similarly, KNN exploits the user-item similarity by finding the K most similar users or items. XGBoost or Random Forest can incorporate more features for prediction.

Models & comparison: We attempted a baseline popularity model, a fine-tuned popularity model, a user-item KNN model & a hybrid model (Table 3.1). The heuristic model had a decent performance and the KNN model failed to outperform the simple heuristic model. Therefore, we focused the core around the heuristic approach and arrived at a hybrid method: it introduced a categories list feature that computes similarity and combined it with the popularity model.

Model	Train Accuracy
Popularity baseline	0.6325
Popularity tuned	0.6334
User-item based KNN	0.6582
Hybrid popularity & similarity	0.8295

Table 3.1 Purchase prediction model comparison

Final model & tuning: The final hybrid method (Fig 3.1) involves 4 key steps: a) item popularity, b) cold start, c) user-item category commonality, and d) user-item category Jaccard similarity. We focused on tuning the following three thresholds: popularity threshold n , commonality threshold k , and Jaccard Index threshold j .

a) Popularity: We assumed that a user will always buy a popular product. As mentioned before, we looked at the product popularity distribution (Fig 2.1) and found that most items have been purchased less than ~ 20 times. We used this as a baseline for grid search and found the optimal value was $n = 18$.

b) Cold start: The popularity approach solves the cold start problem for the 101 new users in the test dataset (Table 2.1) if the item is considered popular. For non-popular items, we assume they will not be purchased.

c) Commonality: We introduced the “categories” list feature to find user-item commonality. We built category lists for each user and item. Using the threshold k , if there are $> k$ common words in the user vs item category lists, we assume the user will buy this item. This is a similarity filtering step but it is necessary before Jaccard because for users or items with many category words, the Jaccard index could be skewed as the union category is large and the

similarity could be underestimated. Having this step addresses this gap.

d) Similarity - Jaccard index: Finally we compute the Jaccard index of user vs item category similarity. We use a threshold j to draw the line and assign the prediction class. We fine tuned the parameter and found the best result occurred when the Jaccard similarity coefficient is $j=0.185$.

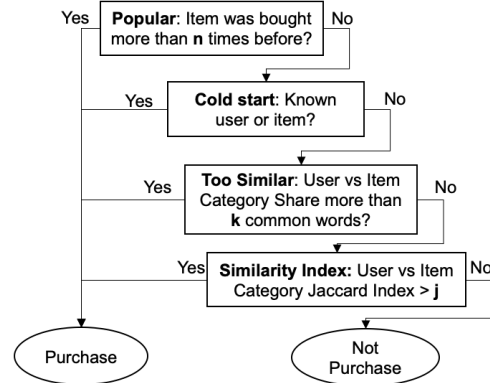


Figure 3.1 Hybrid purchase model illustration

Results: The initial hybrid algorithm achieved an accuracy of 68% and after parameter tunings, we were able to achieve an accuracy of **82.95%** in training and **71.06%** in the final submission.

3.2 Rating Prediction

Approach: Collaborative filtering is common for rating prediction in recommender systems. It can be achieved through similarity or model-based methods. Similarity-based methods analyze the user or item attributes by the level of similarity and predict the ratings by taking the average of the user-item group. Common algorithms include KNN (Eq. 1.1), Jaccard Index, Co-clustering, et cetera. KNN with Mean (Eq. 1.2) introduces a rating mean per item. In the case where insufficient neighbors are present, the prediction will take the mean per user instead of the global mean as KNN Basic. Similarity-based methods are easy to implement, but their performances are more limited than Model-based methods.

$$KNN \text{ Basic: } \hat{r}_{ui} = \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) \times r_{uj}}{\sum_{j \in N_u^k(i)} \text{sim}(i, j)} \quad (\text{Eq. 1.1})$$

$$KNN \text{ with Mean: } \hat{r}_{ui} = \mu_i + \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) \times (r_{uj} - \mu_j)}{\sum_{j \in N_u^k(i)} \text{sim}(i, j)} \quad (\text{Eq. 1.2})$$

Model-based methods transform the rating prediction into a regression or classification problem using machine learning models(i.e. latent-factor with matrix

factorization, decision-tree, etc). As latent factor models usually generate a sparse matrix in recommender systems, common approaches for matrix factorization to facilitate include SVD, PCA, etc.

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u \text{ (Eq. 2)}$$

Co-clustering (Eq. 3) is another similarity-based approach that is similar to the set up of latent-factor (Eq. 2), but instead of using averages per user or item, it takes averages per user or item's clusters.

$$\hat{r}_{ui} = \bar{c}_{ui} + (\mu_u - \bar{c}_u) + (\mu_i - \bar{c}_i) \text{ (Eq. 3)}$$

Models & comparison: We applied both similarity- and model-based approaches. In terms of similarity-based models, we used KNN-inspired algorithms and Co-clustering. To combat the cold start problem, item-based KNN models are used to steer the purchase decision from the similarities between items. KNN Basic and KNN with Mean are implemented with similar general approaches.

$$R = P \Sigma Q^T \text{ (Eq. 4.1)}$$

$$\sum_{r_{ui} \in R} (r_{ui} - \hat{r}_{ui})^2 + \lambda (b_i^2 + b_u^2 + \|q_i\|^2 + \|p_i\|^2) \text{ (Eq. 4.2)}$$

Latent-factor with SVD is the model-based approach we applied. Latent-factor models attempt to represent user-item affinity with a low-dimensional matrix so that both user's preferential information and item's property are included for prediction. SVD helps implement the idea onto the sparse user-item matrix by factorization (Eq. 4.1) and regularization (Eq. 4.2).

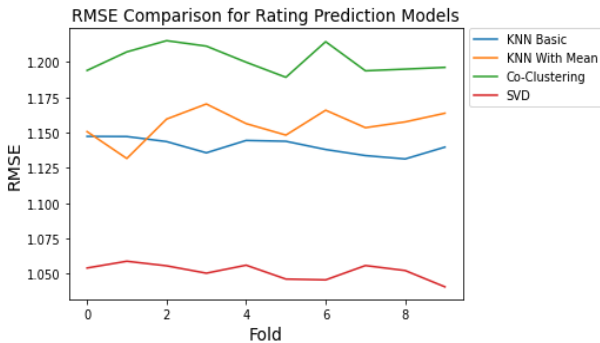


Figure 3.2 Rating prediction models result comparison

We utilized the **Surprise** library^[4] to implement the aforementioned models. RMSE is evaluated using 10-fold cross validation. We found the SVD model performed the best for rating prediction (Fig 3.2).

Final model and tuning: The SVD model was used as the basis prior to final tuning. As stated in Eq. 4.2, the key parameters in this model are the user and item bias, regularization terms, and learning rate. We used GridSearchCV class to find the optimal parameters: ***n_factors=1, reg_bu=0.002, reg_all=0.6, lr_all=0.005***. To assess the impact of review time when splitting the training data into 80/20 sets, we tried both random split and sorting by review time. We found sorting the review time gave a slightly better result in the final accuracy.

Results: Our initial SVD model achieved **1.10796** MSE and after tuning our parameters, we reduced the error to **1.10479** in the final submission.

3.3 Category Prediction

Approach: This is a classic NLP multi-class text classification problem. Each item in the training dataset is labeled (category) based on the review text. We predicted the product category using review text in the test data. This entailed two steps: 1) text processing and word vectorization of review text and 2) category classification. Some popular methods for text vectorization include using bags of words, TF-IDF, and Word2vec. Classification models include Logistic regression, NaiveBayes, XGBoost, and SVM. Convolution neural networks i.e. LSTM and BERT are also used to solve NLP classification problems.

Models & comparison: We tried different methods such as Bag of Words, TF-IDF with ngrams (1,1) and (1,2) and Word2vec to preprocess and vectorize the text; we then attempted various classification models including CNN models for category prediction. We also noted that there was a skewness in the data with the majority of the labels classified in categories 0 and 1 (Table 3.2).

0: Women	1: Men	2: Girls	3: Boys	4: Babies
141398	51416	2976	2329	1881

Table: 3.2 Category matrix

For the text vectorization step, we attempted a number of preprocessing techniques such as stemming and lemmatization to clean the review text and found that the inbuilt preprocessing from TF-IDF Vectorizer resulted in better accuracy. We also tried glove embedding using stanford^[6] glove corpus. For the classification model selection, we used cross validation with 5 folds to get an insight of how the

models generalize to an independent dataset. Our results are in Table 3.3. In addition, we tried the LSTM model using Keras libraries with Softmax activation. Model accuracies are depicted below:

Model	Accuracy
XGBoost	0.816
SVM	0.836
NaiveBayes	0.814
Logistic Regression	0.843
LSTM (CNN)	0.706

Table: 3.3 – Category model comparison

Final model & tuning: The TF-IDF vectorizer with a Logistic Multinomial regression model had the best accuracy, **84.6%**, of all the models. We used this model and tuned the hyper parameters to get the best results for our dataset. We focused on three key parameters for tuning:

Solver: It helped pick the right algorithm; for a multiclass model, sag and saga solvers performed the best with the **sag solver** performing marginally better.

C Inverse of regularization strength: We tuned C from 1 to 3.6 in 0.2 intervals and concluded that a **value of 2.9** was optimal for this dataset.

Penalty: We used **l2 penalties** and used **300 iterations** to achieve optimal results for our dataset.

Results: We achieved an accuracy of **84.3%** after the hyper parameter tuning and our performance on the final test dataset has an accuracy of **84.871%**.

4. DISCUSSION & CONCLUSION

Our proposed models gave considerably good results. For purchase prediction, the hybrid similarity and popularity model outputs an **accuracy of 71.06%** on final test data. Our rating prediction used the latent factor approach with SVD and parameter tuning to achieve an **MSE of 1.10479** with the final test data. Our category prediction used the TF-IDF vectorizer and Logistic Multinomial regression model which accomplished an **accuracy of 84.87%** in the final test data.

Other approaches that can be considered for further improvement can include introducing more creativity in decomposing the data structure and more advanced algorithms to increase the prediction power. In Li-An Yang's approach^[8], the author discussed how the ratio of generated negative dataset (non-purchase) could

impact the purchase prediction result. Further heuristic filtering can also be applied to better handle cold start issues.

TransRec^[3] (Translation-based Recommendation) is another approach to advance the prediction. Instead of interpreting the task as classification, TransRec operates users as "translation vectors" onto the "transition space" embedded by item sequences. Alternatively, to advance from collaborative filtering, heuristics filtering^[1] can be implemented on the data set to adopt a context-aware recommender system.

REFERENCES

- [1] A. Werner-Stark and Z. Nagy, "A Heuristic Method to Recommendation Systems," 2020 6th International Conference on Information Management (ICIM), 2020, pp. 200-204, doi: 10.1109/ICIM49319.2020.244698.
- [2] Borah, Chinmoy. "[Recommendation System Using Collaborative Filtering](#)." *Medium*, Analytics Vidhya. 11 Nov, 2020.
- [3] He, Ruining, *et al.* "[Translation-based Recommendation](#)." *Arxiv*, Cornell University. 8 Jul, 2017.
- [4] Hug, Nicolas. "[Surprise documentation](#)." 2015.
- [5] Kumar, Dr. Vaibhav. "[Singular Value Decomposition \(SVD\) & Its Application in Recommender System](#)." *Analytics India Magazine*. 26 Mar. 2022.
- [6] Pennington, Jeffre, *et al.* "[GloVe: Global Vectors for Word Representation](#)." *The Stanford NLP Group*. Aug 2014.
- [7] Tjokro, Moorissa. "[How to Build a Recommendation System for Purchase Data \(Step-by-Step\)](#)." *Medium*, DataDrivenInvestor. 15 Oct. 2018.
- [8] Yang, An-Li. "[Amazon Purchase Prediction](#)." Fall, 2018.
- [9] Yeung, Albert Au. "[Matrix Factorization: A Simple Tutorial and Implementation in Python](#)." *quuxlabs*, September 16, 2010.
- [10] Zhou, Yifan. "[Bayesian-Personalized-Ranking](#)." *GitHub*. 28 Dec, 2018.